

HYDRA论文内容总结第四版

现有的时间序列数据异常子序列的定位方法中有两类比较具有代表性的方法：

分别是discord-based（基于最近邻距离）（比如MP）和clustering-based（聚类结构）（比如KMeansAD和NormA）

HYDRA算法的目的：

更加准确地找出一系列的时间序列数据中的异常子序列

现有两类代表性方法的问题：

1.discord-based算法通过对每个窗口寻找其最近邻并将两者的距离作为该窗口的异常分数，最近邻距离越大，窗口越有可能异常。但是对于多个异常具有相似形状时，他们会互相成为最近邻，多个形状相似的异常窗口可能互相成为最近邻，从而降低他们的异常分数，异常值就会被忽略。

所以这一个算法只适合检测缺乏相似邻居的孤立异常，但当多个异常窗口彼此高度相似时，它们可能互相成为最近邻，从而造成 nearest-neighbor suppression，使异常分数下降。

（无监督异常检测）

（如MP:表示成向量，通过计算欧氏空间距离找出与其最近邻的）

2.clustering-based算法主要假设正常数据产生稳定簇，异常值是分散出现的并且远离稳定簇，但是当异常值出现的很频繁或者有固定的结构的时候反而就会被忽略

（如K-means: 1. 设定几簇. 2.初始化几个簇中心. 3.分别计算不同窗口与哪个中心比较近，得到两个新簇以后重新计算中心. 4.不断重复）其问题不仅是重复异常可能形成稳定簇，还包括簇数设定不合适以及异常污染中心

归一化（一种常见的前置处理）的问题：

没有一种固定的归一化方式可以适配所有异常值，前两种算法都需要计算向量的距离，但是归一化的过程会改变窗口之间的距离，进而影响异常分数和最终异常检测结果

HYDRA的优点：

HYDRA 避免依赖 per-window input normalization（窗口输入归一化）

但为了让不同层级的异常分数可比较，会执行 level-wise score normalization（层内分数标准化）

实现方法：

找出在不同层级的具有代表性的序列，将全体数据与筛选出来的子序列进行对比来寻找异常值

参考性子序列的筛选步骤：

- 1.把全体序列切割成多个滑动窗口，每一个窗口形成一个向量， $I(L)$ 代表当前在第L层的窗口索引
- 2.计算出 I 中所有切出来的窗口代表的向量之间的欧氏距离
- 3.标记出每一个向量的最近邻的向量
- 4.比较每个窗口与其最近邻的入度：窗口指向入度更大的一方；如果二者入度相同，则指向编号较小的一方。反复沿指向关系追溯，最终指向同一根节点的窗口形成一个局部的簇
- 5.随后每个组保留一个代表窗口，通常直接保留该组的根节点
- 6.将每一个簇中选择出来的代表向量的窗口索引作为 $I(L+1)$
- 7.再次进行同样的筛选步骤，直到代表集合大小达到目标 K
- 8.计算异常分数，在每一层计算过程都对所有初始窗口进行打分，后续再对不同层得到的异常分数进行处理

计算出不同窗口的映射关系 算法1-BuildGroups(W, I):

- 1.先引入变量 W 和 I ，分别代表所有切割的窗口还有，代表当前的层级所有窗口的序列号
- 2.对于当前的序列 I ，统计其中的每一个序列的最近邻窗口 nn ，并统计每一个窗口的入度 $c[j]$ 的大小

- 3.若 $c[i] > c[nn[i]]$ ，则令 $p[i] = i$ ，即当前窗口暂时保持为自身的 parent；若邻居入度更高，则指向邻居；相同则指向索引较小者
- 4.不断进行压缩直至同一个组的都指向同一个窗口
- 5.返回窗口的对应关系 p （字典变量）

计算q 算法2-SelectRepsAndScore(W,l):

- 1.先调用算法1把他的返回值赋值给变量 p
- 2.分别计算初始所有窗口与这一层级选出的窗口代表集合与每一个初始窗口的欧氏距离，最短距离即为所求的 q ，从 BuildGroups 产生的每一个 group 中选一个 representative，形成 R
- 3.返回一个元组变量 (R, q) ，用来代表这一层筛选出来的代表集合 R 和每一个初始窗口的异常分数 q 的列表

HYDRA算法 算法3-HYDRA:

- 1.将表示所有数据的集合 X 按照窗口长度 m ，步长为 s 切割，得到 N 个窗口，得到 $W (m*s)$ ，设定一个窗口索引数临界值 K
- 2.计算第零层每一个窗口的 q 并将层数设置为 $l=0$
- 3.调用算法2把每一层的针对于每一个初始窗口的 q 的数值存入到每一层对应的列表中
- 4.分层对分数进行归一化
- 5.保留每一个窗口在不同层的 q 经过归一化后最大的那一个值，作为一个列表返回值返回，这样一个窗口只要在某一个分辨率下表现得异常，就不会因为其他层级认为它正常而被平均掉
- 6.将每个滑动窗口的最终异常分数重新映射到原始时间轴；由于多个重叠窗口可能覆盖同一个时间点，因此将这些窗口的异常分数取平均，得到每个时间点的最终异常分数 z

理论基础:

正常模式通常出现得更频繁，并在局部邻域中具有更多支持；异常模式数量较少。逐层分组并只保留局部代表，可以压缩重复信息，使异常代表的数量不增加，同时让占多数的正常主导模式更可能被持续保留

优点:

当有多个相似异常的时候，在筛选的过程中倾向于被分到同一个簇中，并且因为筛选的规则，随着层级加深，在异常稀少和多数一致性假设成立时，异常代表的数量理论上不会增加，而正常的主导模式更可能被持续保留